EE6406 Quiz 3 从零开始复习资料（Lecture 9–11）

Analytic and Ensemble Machine Learning · Simon Liu 部分

> 适用：EE6406 Quiz 3 / CA3
>
> 覆盖：**Lecture 9 – Lecture 11**（Ensemble Learning：决策树、Bagging、Random Forest、Boosting、AdaBoost、Gradient Boosting、XGBoost、LightGBM）
>
> 整理日期：2026-11-03
>
> 资料来源：`6406/week9`、`week10`、`week11` 的官方 slides + 转写；`6406/Quiz3/EE6406_集成学习详细笔记.md`；`6406/Learning_note.md`（Week 9 / Week 11）。
>
> 写法：**假设你从零开始**——先讲“为什么”，再讲公式；每个公式都配一个小例子；最后给 Quiz 3 的题型模板和模拟题。

---

0. Quiz 3 要求与 2 天冲刺路线

0.1 考试要求（先记住）

项目	要求
名称	Quiz 3 / CA3（占 **15%**）
时间	**2026-11-05（周四）20:00**；时长 **50 分钟**；Week 12 第一小时的 RL 内容**不考**
形式	**online + in-class + lockdown browser**；必须到课
题型	**True/False + Multiple Choice + Short Calculations**
范围	**Lecture 9 – Lecture 11**（Ensemble Learning）
不考	**Python notebook 内容**（没有 coding 题）
材料	**closed-book**；按课程信息**没有 A4 reference sheet**（以老师/NTULearn 最终通知为准）
Final 关系	老师明确：Final 中他这部分的范围**也正好是 Lecture 9–11**——复习 Quiz 3 = 同时复习期末这部分

0.2 2 天冲刺路线（今天 11/3，考试 11/5）

时间	内容	目标
Day 1 上午	§ 1 前置概念 + § 2 决策树 + § 3 Bagging	能手算 Gini/entropy/方差下降；能解释 bootstrap 63.2%
Day 1 下午	§ 4 Random Forest + § 5 AdaBoost	能背 RF 两个 randomness；能完成一轮 AdaBoost 权重更新
Day 2 上午	§ 6 Gradient Boosting + GBDT	能算 pseudo-residual、line search、叶更新
Day 2 下午	§ 7 XGBoost + § 8 LightGBM	能算叶权重、Gain；能解释 GOSS/EFB/leaf-wise
Day 2 晚上	§ 10 计算模板 + § 11 模拟题	限时做 15 题，错题回看对应小节
Day 3 考前	§ 12 公式表 + § 13 陷阱 + § 14 checklist	只看薄弱点，不再学新内容

> ⭐ **复习原则**：Quiz 3 是 **T/F + MCQ + 短计算**，不是证明题。**概念边界**（bagging vs boosting、RF vs bagging、AdaBoost vs GBDT、XGBoost vs LightGBM）和**6 个计算模板**（Gini、bootstrap、variance、AdaBoost、GBDT、XGBoost）是重点。

---

1. 从零开始：先建立 6 个基本概念

1.1 监督学习在做什么

你有一批**带答案**的数据：

```
(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)
```

- `x_i`：输入/特征（feature）；
- `y_i`：目标/标签（label/target）；
- 模型 `f(x)`：输入 x，输出预测 `ŷ`；
- **训练**：调整模型参数，让预测 `ŷ` 尽量接近 `y`；
- **测试/泛化**：用没见过的 x，看模型预测得准不准。

1.2 分类 vs 回归

	分类（classification）	回归（regression）
目标 y	**离散类别**（猫/狗、+1/-1）	**连续数值**（房价、温度）
输出	class label 或概率	实数
常见 loss	cross-entropy / 0-1 error	squared error
本讲例子	AdaBoost、分类树、RF 分类	回归树、GBDT 回归

> 决策树、RF、GBDT 都**既能分类也能回归**；区别在叶节点输出什么、loss 用什么。

1.3 损失函数（loss）

Loss 衡量“预测有多错”。训练就是让 loss 变小。

任务	常见 loss
回归	squared error：`L = (y - ŷ)^2`；或 `1/2 (y - ŷ)^2`
二分类	0-1 loss；exponential loss：`L = exp(-yF)`；logistic loss：`L = ln(1 + exp(-yF))`
多分类	cross-entropy：`L = -Σ y_k log p_k`

⭐ **注意**：同一个算法换 loss，推导里的常数会变。例如平方损失写成 `(y-F)^2` 时，负梯度是 `2(y-F)`；写成 `1/2(y-F)^2` 时，负梯度是 `y-F`。**考试要看题目用的定义**。

1.4 过拟合与欠拟合

	训练误差	验证误差	直觉
欠拟合 underfitting	高	高	模型太简单 / 训练不够
合适	低	低	泛化好
过拟合 overfitting	很低（≈0）	高（先降后升）	模型太复杂，记住了 noise

- 单棵很深的决策树 → **高 variance**，容易过拟合；
- 集成方法的两大目标：**降 variance**（bagging/RF）或**降 bias**（boosting）。

1.5 训练集 / 验证集 / 测试集

数据	用途
Training set	学参数
Validation set	调超参数、early stopping、选模型
Test set	最后报告一次，不能用来调参

⭐ 常见错误：用 test set 反复调参 → test 变成 validation，最终性能偏乐观。

1.6 偏差（bias）与方差（variance）

- **Bias**：模型平均预测与真实值的偏离 → 模型是否**太简单**；
- **Variance**：换一份训练集，预测变化多大 → 模型是否**太敏感**；
- **Irreducible noise**：数据本身的随机性，任何模型都消不掉。

⭐ 一句话：

- **Bagging / Random Forest** 主要**降 variance**；
- **Boosting / AdaBoost / GBDT** 主要**降 bias**（但控制不好会升 variance）。

1.7 符号表（后面一直用）

符号	含义
n	样本数
p	特征数
M	基模型/树的数量
K	类别数
T	一棵树的叶子数（leaf count）
w_j	第 j 个叶子的输出权重
γ（gamma）	XGBoost 的叶子数惩罚 / GBDT 的步长（看上下文）
λ（lambda）	L2 正则系数
η（eta）	learning rate / shrinkage
ρ（rho）	模型之间的相关系数
ε（epsilon）	AdaBoost 的加权错误率
α（alpha）	AdaBoost 的弱分类器权重

---

2. Lecture 9（上）：决策树 Decision Tree

2.1 树是什么

决策树 = 一串 **if-else 问题**：

```
如果 x1 < 5:
    如果 x2 < 3: → 类别 A
    否则:        → 类别 B
否则:
    如果 x3 > 0: → 类别 C
    否则:        → 类别 D
```

- **Root node**：第一个问题；
- **Internal node**：中间的问题；
- **Leaf node**：不再问，输出类别/数值；
- 每条 root→leaf 路径 = 一条规则。

分类树：叶节点输出多数类别，或类别比例作为概率。 **回归树**：叶节点输出该叶内训练样本的**平均值** $\bar{y}$。

2.2 怎么选“最好的问题”？——不纯度（impurity）

在每个节点，尝试所有 feature 和阈值，选一个让子节点**最纯**的分裂。

⭐ 核心公式（分裂收益 / impurity drop）：

```
ΔI = I(parent) - (n_L/n) I(left) - (n_R/n) I(right)
```

- `I`：不纯度（越小越纯）；
- `n_L, n_R`：左右子节点样本数；
- 收益越大越好；等价于**加权子节点不纯度最小**。

2.3 分类树：Gini 不纯度

节点内有 K 类，第 k 类比例 `p_k`：

```
G = 1 - Σ_k p_k^2
```

- 只有一类 → `G = 0`（最纯）；
- 两类各 50% → `G = 1 - 0.5^2 - 0.5^2 = 0.5`（最不纯）；
- 二分类可写成 `G = 2p(1-p)`。

例子（必会）：父节点 10 条，正负各 5：

```
G_parent = 1 - 0.5^2 - 0.5^2 = 0.5
```

分裂后：

- 左：5 条，4 正 1 负 → `G_L = 1 - 0.8^2 - 0.2^2 = 0.32`
- 右：5 条，1 正 4 负 → `G_R = 0.32`

加权：

```
G_after = (5/10)*0.32 + (5/10)*0.32 = 0.32
ΔG = 0.5 - 0.32 = 0.18
```

2.4 分类树：熵（entropy）与信息增益

```
H = -Σ_k p_k log2(p_k)
```

约定 `0 log 0 = 0`。

- 只有一类 → `H = 0`；
- 二分类 50/50 → `H = -0.5 log2 0.5 - 0.5 log2 0.5 = 1 bit`。

同一个例子：

```
H_parent = 1
H_L = H_R = -0.8 log2 0.8 - 0.2 log2 0.2 ≈ 0.7219
H_after = 0.5*0.7219 + 0.5*0.7219 ≈ 0.7219
ΔH = 1 - 0.7219 ≈ 0.2781 bit
```

⭐ **Gini 和 entropy 不能直接比数值**：Gini 的 0.18 和 entropy 的 0.2781 刻度不同；只能在**同一种指标**下比较不同分裂。

2.5 回归树：方差 / 平方误差

叶节点预测 `ȳ`。节点方差：

```
Var(N) = (1/n) Σ_i (y_i - ȳ)^2
```

分裂收益：

```
ΔVar = Var(parent) - (n_L/n) Var(left) - (n_R/n) Var(right)
```

⭐ 为什么用均值？对常数预测 c，最小化 `Σ(y_i - c)^2`，求导得 `c = ȳ`。

2.6 什么时候停止生长

- 最大深度 `max_depth`；
- 节点最少样本 `min_samples_split`；
- 叶节点最少样本 `min_samples_leaf`；
- 最大叶子数 `max_leaf_nodes`；
- 最小不纯度下降 `min_impurity_decrease`。

⭐ 树越深 → 表达能力越强，但越容易过拟合。单棵深树是**高 variance** 模型——这正是集成方法要解决的问题。

2.7 决策树的优缺点

优点	缺点
可解释（if-else 规则）	单棵树高 variance
能处理非线性、feature interaction	对数据小变化敏感
不需要 feature scaling	容易过拟合（深树）
可处理类别/数值特征	贪心分裂，不保证全局最优

---

3. Lecture 9（下）：Bias–Variance、Bootstrap、Bagging

3.1 误差分解（平方损失）

设真实函数 `f(x)`，观测 `Y = f(x) + ε`，`E[ε]=0`，`Var(ε)=σ²`。则：

```
E[(Y - ŷ(x))^2] = (E[ŷ(x)] - f(x))^2 + Var(ŷ(x)) + σ²
                  \_______bias²_______/   \__variance__/   \noise/
```

- **Bias²**：模型平均预测偏离真实函数；
- **Variance**：模型对训练集随机性的敏感度；
- **σ²**：不可约噪声。

⭐ Bagging/RF 主要减小 **variance** 项；Boosting 主要减小 **bias²** 项。

3.2 Bootstrap：有放回抽样

- 原数据 n 条；
- 每次从原数据中**有放回**抽 n 条，形成一个 bootstrap 数据集；
- 同一条可能被抽多次，也可能一次都没被抽到。

某条样本一次都没被抽中的概率：

```
P(not selected in one draw) = 1 - 1/n
P(not selected in n draws)  = (1 - 1/n)^n → e^(-1) ≈ 0.368
```

所以 n 很大时：

- **约 36.8%** 的原始样本不在某个 bootstrap 样本中；
- **约 63.2%** 的不同原始样本会出现。

⭐ 例：原数据 [A,B,C,D]，某次 bootstrap 可能是 [A,A,C,D]：A 出现两次，B 没出现。

3.3 Bagging：Bootstrap + Aggregating

Bagging = Bootstrap Aggregating：

1. 生成 M 个 bootstrap 数据集；
2. 每个数据集独立训练一个基模型；
3. 回归：平均预测；分类：多数投票/概率平均。

回归：

```
f_bag(x) = (1/M) Σ_m f_m(x)
```

⭐ 直觉：单棵深树对训练集变化很敏感；平均多棵不完全相同的树，可以抵消一部分波动。

3.4 平均为什么能降 variance（必考公式）

假设每个模型方差都是 σ^2，任意两个模型相关系数都是 ρ：

```
Var( (1/M) Σ_m f_m ) = σ² ( ρ + (1-ρ)/M )
```

情况	结果
$\rho = 0$（完全独立）	方差 = σ^2/M（平均很有效）
$\rho = 1$（完全相关）	方差 = σ^2（平均无效）
$0 < \rho < 1$	方差 → $\rho\sigma^2$（M 很大时）

例子：$\sigma^2=1$，M=100，$\rho=0.2$：

```
Var = 1 * (0.2 + 0.8/100) = 0.208
```

⭐ 关键：**增加模型数只能降低 (1-ρ)/M 这一项；ρ 本身要靠“让模型更不同”来降低**——这就是 Random Forest 随机选特征的目的。

3.5 Stacking（简单了解）

- 用多个基模型的预测作为新特征，再训练一个**元模型**（meta-model）来组合；
- 训练元模型要用**折外预测（out-of-fold）**，避免基模型对自己训练数据过拟合导致泄漏；
- Quiz 可能只考“stacking 是什么/为什么用 out-of-fold”，不会考复杂实现。

---

4. Lecture 10（上）：Random Forest

4.1 Random Forest = Bagging + 特征随机

普通 Bagging 树：每棵树用 bootstrap 样本，但**每次分裂考虑所有特征**。 Random Forest 增加第二层随机性：

1. 每棵树用 bootstrap 样本；
2. **每次节点分裂时，从全部 p 个特征中随机选一部分候选特征**（分类常用 `√p`，回归常用 `p/3`，但不是通用最优）；
3. 只在这些候选特征中选最佳分裂；
4. 多棵树投票/平均。

⭐ **两个 randomness**：

- **行随机**：bootstrap 采样；
- **列随机**：每次分裂随机选特征。

目的：降低树之间的 correlation ρ → 由 §3.4，ρ 越小，bagging 降 variance 越有效。

4.2 森林的预测

回归：

```
f_RF(x) = (1/M) Σ_m T_m(x)
```

分类（硬投票）：

```
ŷ = argmax_k Σ_m 1[T_m(x) = k]
```

有些实现先平均各类概率再取 argmax（软投票）；硬投票和软投票不一定相同。

4.3 OOB（Out-of-Bag）评估

- 对每条训练样本，只使用**没有把它抽进 bootstrap** 的树来预测；
- 聚合这些树的预测，得到该样本的 OOB 预测；
- 用所有有 OOB 预测的样本算误差 → **OOB error**。

⭐ 因为每个 bootstrap 约 36.8% 样本没被抽到，所以每条样本都有约 36.8% 的树没“见过”它，可做内部验证。OOB 不是所有场景下独立 test set 的替代品（时间序列、数据泄漏等要额外设计）。

4.4 Feature importance

方法	做法	注意
Impurity-based	累计某特征分裂带来的加权不纯度下降	高基数特征可能虚高
Permutation	打乱某特征，看验证性能下降多少	相关特征会互相替代，可能低估重要性

⭐ 重要：feature importance **不是因果效应**；两个高度相关的强特征会“抢”重要性。

4.5 RF 超参数

参数	作用
`n_estimators`	树的数量；几百棵后收益递减
`max_depth`	单棵树深度
`min_samples_split` / `min_samples_leaf`	控制叶节点大小
`max_features`	每次分裂候选特征数（RF 的核心随机性）
`bootstrap`	是否有放回采样
`oob_score`	是否用 OOB 估计

---

5. Lecture 10（下）：Boosting 与 AdaBoost

5.1 Bagging vs Boosting（必背）

	Bagging / RF	Boosting
训练方式	多模型**独立**，可并行	**顺序**，新模型依赖前面的模型
差异来源	bootstrap / 随机特征	根据前面的误差/梯度调整
基模型	可较深	通常浅树/弱学习器
主要作用	**降 variance**	**降 bias**（过多轮可能升 variance）
过拟合	相对不易（平均）	需要 learning rate/early stopping 控制

5.2 AdaBoost 算法（二分类，$y \in \{-1,+1\}$）

输入：训练集 `(x_i, y_i)`，弱学习器 `h(x)`，轮数 M。

1. 初始化样本权重：

```
w_i = 1/n,  i = 1..n
```

1. 对 m = 1..M：
2. 在当前权重下训练弱分类器 `h_m(x)`；
3. 计算**加权错误率**：

```
ε_m = Σ_i w_i * 1[ h_m(x_i) ≠ y_i ]
```

- 计算弱分类器权重：

```
α_m = (1/2) ln( (1 - ε_m) / ε_m )
```

- 更新样本权重：

```
w_i ← w_i * exp( -α_m * y_i * h_m(x_i) )
```

- 归一化：`w_i ← w_i / Z_m`，使 `Σ w_i = 1`。

1. 最终分类：

```
H(x) = sign( Σ_m α_m h_m(x) )
```

5.3 为什么 α 长这样？（指数损失推导）

AdaBoost 等价于在指数损失 `L(y,F) = exp(-yF)` 上做前向分步加法模型。当前正确样本权重和 = `1-ε`，错误样本权重和 = `ε`。加入 `α h(x)` 后：

```
J(α) = (1-ε) e^(-α) + ε e^(α)
```

求导：

```
J'(α) = -(1-ε)e^(-α) + εe^(α) = 0
=> e^(2α) = (1-ε)/ε
=> α* = (1/2) ln( (1-ε)/ε )
```

`J''(α) > 0`，所以是极小值。

⭐ 直觉：

- ε < 0.5（比随机好）→ α > 0；
- ε = 0.5 → α = 0（这个 learner 没用）；
- ε → 0 → α → ∞（完美 learner，权重很大；程序要数值保护）。

5.4 为什么错分样本权重变大？

- 分类正确：`y_i h_m(x_i) = +1` → 权重乘 `e^(-α)`（变小）；
- 分类错误：`y_i h_m(x_i) = -1` → 权重乘 `e^(+α)`（变大）。

例子（必会）：4 条样本，初始权重 0.25；弱分类器错 1 条。

```
ε = 0.25
α = (1/2) ln(0.75/0.25) = (1/2) ln 3 ≈ 0.5493
```

错分样本乘 `e^α = √3 ≈ 1.732`；正确样本乘 `e^(-α) = 1/√3 ≈ 0.577`。归一化后：

- 错分样本：**0.5**

- 3 条正确样本：各 **1/6 ≈ 0.1667**

⭐ 下一轮会更关注错分样本；但也可能反复关注**错误标签/异常点** → AdaBoost 对噪声敏感。

5.5 指数损失与“半个对数几率”

在固定 x，设 `p = P(Y=+1|x)`。条件期望指数损失：

```
p e^(-F) + (1-p) e^(F)
```

对 F 求导令零：

```
-pe^(-F) + (1-p)e^(F) = 0
=> F*(x) = (1/2) ln( p/(1-p) )
```

⭐ 在 `exp(-yF)` 约定下，最优分数是**半个 log-odds**；有些讲义/教材省略 1/2，取决于 F 的定义。考试若考这个细节，按题目定义走。

5.6 AdaBoost 的边界情况

- `ε = 0.5` → α = 0（不比随机好）；
- `ε = 0` → α → ∞（完美分类器；实际要提前停止/数值保护）；
- `ε > 0.5` → α < 0（比随机差；通常翻转预测或停止）；
- 噪声/异常点会被反复加权 → 可能过拟合。

---

6. Lecture 11（上）：Gradient Boosting 与 GBDT

6.1 从“加性模型”开始

Gradient Boosting 把模型写成多个弱学习器的和：

```
F_m(x) = F_{m-1}(x) + η * γ_m * h_m(x)
```

- `h_m`：第 m 轮的弱学习器（通常是浅回归树）；
- `γ_m`：步长（line search 得到）；
- `η`：learning rate / shrinkage（$0 < \eta \leqslant 1$）。

⭐ 每一步不是重新训练一个独立模型，而是**在当前模型上再加一个小模型**。

6.2 伪残差（pseudo-residual）

一般 loss `L(y, F(x))` 对 F 可微。定义：

```
r_i^(m) = - ∂L(y_i, F) / ∂F |_{F = F_{m-1}(x_i)}
```

- 它是 loss 下降最快的方向（负梯度）；
- 用回归树 `h_m(x)` 去拟合 `{(x_i, r_i^(m))}`；
- 新树输出“往哪个方向调整”，再加到当前预测上。

平方损失 `L = (y - F)^2`：

```
∂L/∂F = -2(y - F)
r = -∂L/∂F = 2(y - F)
```

例：y=5，F=3 → r = 4。

logistic loss `L = ln(1 + exp(-yF))`，y ∈ {-1,+1}：

```
r = y / (1 + exp(yF))
```

例：y=+1，F=0 → r=0.5；y=-1，F=1 → r≈-0.269。

⭐ 注意：如果 loss 写成 `1/2(y-F)^2`，则 r = y-F（少一个因子 2）。**看题目定义**。

6.3 为什么叫“函数空间梯度下降”？

- 普通梯度下降：对**参数** θ 求导；
- Gradient Boosting：对**模型函数 F** 求导（functional derivative）；
- 每个弱学习器是函数空间中的一个“基方向”；
- 精确的函数梯度是无限维的，实际用样本上的 pseudo-residual 做近似；
- 所以叫“在函数空间做梯度下降”。

6.4 GBDT 算法（必背）

1. 初始化常数预测 `F_0(x)`，使训练 loss 最小（平方损失下 = `ȳ`）；
2. 对 m = 1..M：
3. 计算每条样本的 pseudo-residual `r_i^(m)`；
4. 拟合回归树 `h_m(x)` 到 `{(x_i, r_i)}`；
5. 计算步长 `γ_m`（全局或每个叶子分别算）；
6. 更新 `F_m(x) = F_{m-1}(x) + η γ_m h_m(x)`；
7. 输出 `F_M(x)`。

6.5 步长 γ 怎么算?

全局线搜索（平方损失）：

```
γ* = Σ_i h_i (y_i - F_{m-1}(x_i)) / Σ_i h_i^2
```

例：残差 `[2,2,3]`，树输出 `[1,1,1]`：

```
γ* = (2+2+3) / (1+1+1) = 7/3 ≈ 2.33
```

叶节点单独更新（平方损失）：

叶内最优更新 = 该叶内残差的平均值。

例：叶内残差 `[2,3,4]` → `γ_r = 3`。

⭐ 全局 γ 是一个数；region-specific γ 是每个叶子一个数，更灵活但更贵。

6.6 正则化与调参

手段	作用
learning rate / shrinkage η	每轮只走一小步；小 η 需要更多树
树的数量 M	太少欠拟合，太多过拟合
max_depth / num_leaves	控制单棵树复杂度
min_samples_leaf	防止叶节点太小
row subsampling	每棵树用部分样本
column subsampling	每次分裂用部分特征
early stopping	验证集不改善就停

6.7 Bias–variance 视角

- Boosting 顺序拟合残差 → **bias 快速下降**；
- 不加正则、树太多 → **variance 上升/过拟合**；
- 所以需要 shrinkage、subsampling、树约束、early stopping。

---

7. Lecture 11（中）：XGBoost

7.1 目标函数

XGBoost 在 GBDT 基础上加**显式正则**：

```
J = Σ_i L(y_i, ŷ_i) + Σ_m Ω(f_m)
Ω(f) = γ T + (λ/2) Σ_j w_j^2
```

- `T`：一棵树的叶子数；
- `w_j`：第 j 个叶子的输出；
- `γ`：每增加一个叶子的惩罚（控制树复杂度）；
- `λ`：叶子权重的 L2 正则（防止权重过大）。

7.2 二阶 Taylor 展开

加入新树 `f` 后，对 loss 在当前预测 `F_{m-1}` 处展开：

```
L(y, F + f) ≈ L(y, F) + g f + (1/2) h f^2
g = ∂L/∂F |_{F_{m-1}}
h = ∂²L/∂F² |_{F_{m-1}}
```

- `g`：梯度（斜率）；
- `h`：Hessian（曲率）；
- XGBoost 同时用一阶和二阶信息 → 收敛更快、更准。

7.3 最优叶子权重（必考）

对一个叶子 j，样本集合 `I_j`，记：

```
G_j = Σ_{i∈I_j} g_i
H_j = Σ_{i∈I_j} h_i
```

叶子目标：

```
J_j(w) = G_j w + (1/2)(H_j + λ) w^2
```

求导令零：

```
G_j + (H_j + λ) w = 0
=> w_j* = - G_j / (H_j + λ)
```

平方损失例子 `L=(y-F)^2`：

- `g_i = 2(F_i - y_i)`
- `h_i = 2`
- 三个样本 y=[5,7,9], F=[3,4,5] → `g=[-4,-6,-8]`，`G=-18`；`H=6`
- λ=0 → w=*3*；*λ=2* → *w*=18/8=2.25；λ=10 → w*=18/16=1.125。

⭐ λ 越大，叶权重越向 0 收缩 → 正则化更强。

7.4 分裂增益 Gain（必考）

父节点分成左/右：

```
G = G_L + G_R
H = H_L + H_R
Gain = (1/2)[ G_L^2/(H_L+λ) + G_R^2/(H_R+λ) - G^2/(H+λ) ] - γ
```

- 只有 **Gain > 0** 才分裂；
- γ 越大 → 分裂门槛越高 → 树越简单。

数值例子：`G_L=10, H_L=8; G_R=-4, H_R=5; λ=0`：

```
G = 6, H = 13
pre-penalty = 0.5 * (100/8 + 16/5 - 36/13)
            = 0.5 * (12.5 + 3.2 - 2.769) ≈ 6.465
γ=5 → Gain ≈ 1.465 > 0 → 接受
γ=7 → Gain ≈ -0.535 < 0 → 拒绝
```

7.5 缺失值与工程优化

- XGBoost 把 missing 当**单独类别**，学习一个 **default direction**：每个 split 试“missing 全去左/全去右”，选 gain 大的方向；
- 不需要手动 imputation；

- 工程：column block 压缩、histogram 分裂、cache 优化、并行/分布式、GPU、AllReduce。

7.6 XGBoost 超参数

参数	作用	典型范围
`eta` / learning_rate	步长	0.01–0.2
`n_estimators`	树数	100–1000（early stopping）
`max_depth`	深度	3–7
`subsample`	行采样	0.6–0.8
`colsample_bytree`	列采样	0.5–0.8
`gamma`	最小 split gain	0–5
`lambda`	L2 on leaf weights	1–10
`min_child_weight`	叶最小样本权重和	1–10

---

8. Lecture 11（下）：LightGBM

8.1 Leaf-wise 生长

方式	做法
Level-wise	一层一层扩展，所有节点都分裂
Leaf-wise (best-first)	每次选**当前 split gain 最大**的单个叶子分裂

- 优点：每次分裂的 loss reduction 更大，同样精度需要更少树；
- 缺点：小数据容易过拟合（用 `num_leaves`、`max_depth`、`min_data_in_leaf` 控制）。

⭐ **叶子计数**：一棵二叉树从 1 个叶子开始，每次分裂增加 1 个叶子；从 1 到 4 个叶子需要 **3 次分裂**（`4-1=3`）。讲义 L11 p62 的例子里写了 4 次分裂却写“得到 4 个叶子”，这是计数不一致；考试按“每次分裂 +1 个叶子”算。

8.2 GOSS（Gradient-based One-Side Sampling）

直觉：大梯度样本对分裂更重要；小梯度样本贡献小，可以采样。

步骤：

1. 按 `|g_i|` 从大到小排序；
2. 保留 top `a` 比例的大梯度样本；
3. 从剩余样本中随机抽 `b` 比例（**以总样本数为基数**）；
4. 对小梯度样本乘以 `(1-a)/b` 放大，保持总和近似不变。

例子：10 条样本，a=0.4，b=0.2：

- 保留 4 条大梯度；
- 剩余 6 条中抽 2 条（2/6=1/3 概率）；

- 放大因子 `(1-0.4)/0.2 = 3`；
- 若梯度为 `[9,8,7,6,5,4,3,2,1,0.5]`，保留 `[9,8,7,6]`，抽到 5 和 3 → `[15,9]`；训练用 `[9,8,7,6,15,9]`。

⭐ 比例口径：若 b 指“剩余样本中的比例”，放大因子是 `1/b`；讲义文字有歧义，按题目定义走。

8.3 EFB（Exclusive Feature Bundling）

- 互斥的稀疏特征（很少同时非零）可以合并成一个 bundle；
- 用 **offset** 区分原特征；
- 例：f1=8, f2=5, f3=3, f4=9；offsets 0,10,20,30 → bundle 值 `8,15,23,39`；
- 好处：减少 feature 数/内存（最高约 80%），信息不丢失（严格互斥时）。

8.4 原生类别特征

- 不需要 one-hot；
- 每个 category 当作一个 bin；按 **mean target** 排序，找最优二分；
- 例：A 均值 10.5，B 12.5，C 4.5 → 排序 B, A, C → 最优 split {A,B} vs {C}（均值 11.5 vs 4.5）。

8.5 缺失值

- 与 XGBoost 相同：missing 当单独类别，split 时试去左/去右，选 gain 大的方向；不需要手动 imputation。

8.6 LightGBM 超参数

参数	作用	典型范围
`num_leaves`	每棵树最大叶子数	31–255
`learning_rate`	步长	0.01–0.2
`n_estimators`	树数	100–1000
`max_depth`	深度	3–7
`min_data_in_leaf`	叶最小样本数	10–50
`feature_fraction`	列采样	0.5–0.8
`bagging_fraction`	行采样	0.6–0.8
`min_gain_to_split`	最小 split gain	0–5
`lambda_l2`	L2 on leaf weights	0–10

8.7 XGBoost vs LightGBM（必背）

特征	XGBoost	LightGBM
Tree growth	**Level-wise**	**Leaf-wise**
Missing values	Sparse-aware default direction	同
Categorical	需要 encoding	**原生支持**
Sampling	Row/column subsample	**GOSS + EFB** + subsampling
Split pruning	γ threshold	γ + **num_leaves**
Memory	Moderate	**High**（EFB 降维）
Speed	并行化后快	**更快**（leaf-wise + sampling）
典型场景	平衡精度/稳定性	**超大数据、稀疏特征**

9. 模型对比与选择

方面	决策树	随机森林	AdaBoost	GBDT / XGBoost / LightGBM
结构	单棵树	多棵随机树	弱分类器加权和	多轮加性树
训练	递归分裂	多树可并行	串行	串行（轮内可并行）
主要作用	—	**降 variance**	**降 bias**	**降 bias + 控制 variance**
风险	过拟合/欠拟合	树间相关	噪声敏感	过拟合、调参复杂
解释性	单棵浅树直观	整体较复杂	整体较复杂	多树组合复杂

⭐ 没有“永远最好”的算法；用**同一数据划分、同一 metric、同一算力预算**比较。

10. Quiz 3 计算题模板（短计算必看）

模板 1：Gini / entropy 分裂收益

步骤：

1. 算父节点 `G` 或 `H`；
2. 算左右子节点；
3. 加权：`(n_L/n)*I_L + (n_R/n)*I_R`；
4. 收益 = 父 - 加权子。

例：父 10 条 5/5；子 4/1 和 1/4 → Gini 0.5 → 0.32，收益 0.18；entropy 1 → 0.7219，收益 0.2781。

模板 2：Bootstrap 概率

```
P(not selected) = (1 - 1/n)^n ≈ e^(-1) ≈ 0.368
P(selected at least once) ≈ 0.632
```

例：n=1000 → 0.3677。

模板 3：平均模型的 variance

```
Var(avg) = σ² ( ρ + (1-ρ)/M )
```

例：σ^2=1，ρ=0.2，M=10 → 0.28；M=100 → 0.208。

模板 4：AdaBoost 一轮

```
ε = Σ w_i * 1[wrong]
α = 1/2 ln((1-ε)/ε)
wrong weight *= e^α
correct weight *= e^(-α)
normalize
```

例：4 条，ε=0.25 → α≈0.5493；wrong=0.5，correct=1/6。

模板 5：GBDT 伪残差 / 叶更新

- 平方损失 `L=(y-F)^2` → `r=2(y-F)`；
- 若 `L=1/2(y-F)^2` → `r=y-F`；
- 叶内最优更新（平方损失）= 叶内残差均值；
- 全局 γ = `Σ h_i(y_i-F_i) / Σ h_i²`。

模板 6：XGBoost 叶权重 / Gain

```
w* = -G/(H+λ)
Gain = 1/2[G_L²/(H_L+λ) + G_R²/(H_R+λ) - G²/(H+λ)] - γ
```

例：G=-18,H=6,λ=2 → w*=2.25；G_L=10,H_L=8,G_R=-4,H_R=5,λ=0 → pre-penalty≈6.465。

模板 7：LightGBM 叶子数 / GOSS / EFB

- 4 个叶子需要 **3 次分裂**；
- GOSS：top a%，剩余抽 b%，权重 `(1-a)/b`；
- EFB：互斥特征 + offsets 合并。

---

11. 模拟题（15 题，含答案）

> 先遮住答案做；Quiz 3 是 T/F + MCQ + short calculations，下面按这三类混合。

A. True/False（5 题）

T1. Bagging 主要通过降低 variance 来改善模型；Boosting 主要通过降低 bias。 **T2.** Random Forest 在每次节点分裂时使用全部特征，因此树之间相关性很高。 **T3.** AdaBoost 中，被错分的样本在下一轮权重会变大。 **T4.** 在 GBDT 中，pseudo-residual 总是等于 `y - F`，与损失函数无关。 **T5.** XGBoost 的叶子权重公式是 `w* = -G/(H+λ)`，λ 越大叶权重越向 0 收缩。

B. Multiple Choice（5 题）

M1. 下列哪种方法主要用于降低 variance？ A. AdaBoost B. Random Forest C. 单棵决策树 D. Gradient Boosting

M2. AdaBoost 的弱分类器权重 α 在加权错误率 ε=0.25 时约为： A. 0.25 B. 0.5493 C. 1.0986 D. 0.6931

M3. XGBoost 相比普通 GBDT 的关键改进不包括： A. 二阶 Taylor 展开 B. 显式正则项 C. 仅使用一阶梯度 D. 缺失值 default direction

M4. LightGBM 的 GOSS 主要目的是： A. 增加树的数量 B. 对大梯度保留、小梯度采样以加速 C. 对类别特征 one-hot D. 用 level-wise 生长

M5. 随机森林的 OOB 样本比例约为： A. 63.2% B. 36.8% C. 50% D. 80%

C. Short Calculations（5 题）

C1. 父节点 10 条样本正负各 5；分裂后左 4 正 1 负，右 1 正 4 负。求 Gini 收益。

C2. n=1000 的 bootstrap，某样本一次都没被抽中的概率约为多少？

C3. 每个模型 variance=1，任意两模型 correlation ρ=0.2，平均 10 个模型和 100 个模型的 variance 各是多少？

C4. AdaBoost 4 条样本初始权重 0.25，弱分类器错 1 条。求 ε、α、归一化后错分样本权重。

C5. XGBoost 一个叶子 G=-18，H=6，λ=2。求最优叶权重；若 λ=0 呢？

答案与简解

题	答案	简解
T1	✅	bagging 降 variance；boosting 降 bias
T2	❌	RF 每次分裂只随机选一部分特征
T3	✅	错分样本乘 `e^α`，权重变大
T4	❌	只有平方损失才是 `2(y-F)`（或 `y-F`）；一般 loss 是负梯度
T5	✅	`w*=-G/(H+λ)`；λ 越大越小
M1	**B**	RF 是 bagging 的代表
M2	**B**	`0.5 ln(0.75/0.25)=0.5 ln3≈0.5493`
M3	**C**	XGBoost 用一阶+二阶，不是仅一阶
M4	**B**	GOSS = 大梯度保留 + 小梯度采样
M5	**B**	约 36.8% 未抽中（63.2% 出现）
C1	0.18	父 0.5；子各 0.32；0.5-0.32=0.18
C2	≈0.3677	`(1-1/1000)^1000`
C3	0.28；0.208	`1*(0.2+0.8/10)=0.28`；`1*(0.2+0.8/100)=0.208`
C4	ε=0.25；α≈0.5493；错分 0.5	错分乘 √3，正确乘 1/√3，归一化
C5	λ=2 → 2.25；λ=0 → 3	`18/(6+2)=2.25`；`18/6=3`

12. 公式记忆表（闭卷）

> 6406 Quiz 3 是 closed-book；下面这些要能**默写**。

12.1 树与不纯度

```
Gini:     G = 1 - Σ_k p_k^2
Entropy:  H = -Σ_k p_k log2 p_k
Split:    ΔI = I(parent) - (n_L/n) I(L) - (n_R/n) I(R)
Reg tree: Var(N) = (1/n) Σ (y_i - ȳ)^2
```

12.2 Bootstrap / Bagging

```
P(not selected) = (1 - 1/n)^n → e^(-1) ≈ 0.368
Unique ≈ 0.632
Bagging avg: f_bag = (1/M) Σ f_m
Var(avg) = σ² ( ρ + (1-ρ)/M )
```

12.3 AdaBoost

```
ε_m = Σ_i w_i 1[h_m(x_i) ≠ y_i]
α_m = (1/2) ln((1-ε_m)/ε_m)
w_i ← w_i exp(-α_m y_i h_m(x_i)) / Z_m
H(x) = sign(Σ_m α_m h_m(x))
```

12.4 GBDT

```
F_m = F_{m-1} + η γ_m h_m
r_i = -∂L/∂F |_{F_{m-1}}
Squared: r = 2(y-F)  (or y-F if L=1/2(...)²)
Global γ* = Σ h_i (y_i - F_i) / Σ h_i²
Leaf γ_r = mean residual in leaf (squared loss)
```

12.5 XGBoost

```
J = Σ L + Σ [γT + (λ/2)Σ w_j²]
g_i = ∂L/∂F; h_i = ∂²L/∂F²
w_j* = -G_j/(H_j+λ)
Gain = 1/2[G_L²/(H_L+λ)+G_R²/(H_R+λ)-G²/(H+λ)] - γ
```

12.6 LightGBM

```
Leaf-wise: split leaf with max gain; 4 leaves = 3 splits
GOSS: keep top a%; sample b% of rest; scale (1-a)/b
EFB: bundle mutually exclusive sparse features with offsets
Categorical: sort by mean target, find best binary split
```

---

13. 高频陷阱（考前必看）

#	陷阱	正确说法
1	Bagging 降 bias	❌ Bagging 主要降 **variance**
2	Boosting 一定比 bagging 好	❌ 取决于数据/调参；boosting 对噪声敏感
3	RF = bagging（完全一样）	❌ RF 多了**每次分裂随机选特征**
4	RF 的树之间完全独立	❌ bootstrap + 特征随机降低相关性，但不为 0
5	OOB = 独立 test set	❌ 是内部估计；时间序列/泄漏要额外验证
6	Gini 和 entropy 可以比数值	❌ 刻度不同，只能同指标比较
7	AdaBoost 错分样本权重变小	❌ 变大（乘 `e^α`）
8	AdaBoost 的 α 只由样本数决定	❌ 由加权错误率 ε 决定
9	GBDT 的 pseudo-residual 总是 y-F	❌ 一般 loss 是负梯度；平方损失才是 2(y-F)
10	GBDT 的 γ 总是 1	❌ 由 line search 得到
11	XGBoost 只用一阶梯度	❌ 用一阶 g + 二阶 h
12	XGBoost 的 λ 越大叶权重越大	❌ λ 越大，`w*=-G/(H+λ)` 越小
13	XGBoost 的 γ 越大树越深	❌ γ 越大，分裂门槛越高，树越简单
14	LightGBM 是 level-wise	❌ leaf-wise（best-first）
15	4 个叶子需要 4 次分裂	❌ 从 1 个叶子开始，4 个叶子需要 3 次分裂
16	GOSS 保留所有小梯度样本	❌ 只采样一部分并放大
17	EFB 对任何特征都无损	❌ 需要互斥/近似互斥；有冲突要处理
18	LightGBM 一定比 XGBoost 准	❌ 大数据更快；小数据精度可能接近甚至 XGBoost 更好
19	单棵深树 variance 低	❌ 单棵深树 variance 高
20	决策树不需要 feature scaling，所以永远最好	❌ 单棵树高 variance，容易过拟合

14. 考前 1 小时 checklist

- ☐ 能默写 Gini、entropy、split gain。
- ☐ 能解释 bootstrap 63.2% / OOB 36.8%。
- ☐ 能默写 `Var(avg)=σ²(ρ+(1-ρ)/M)` 并代数字。
- ☐ 能背 RF 的两个 randomness。
- ☐ 能背 AdaBoost 的 ε、α、weight update，并做 4 条样本的例子。
- ☐ 能区分 AdaBoost 的 α（弱分类器权重）和 GBDT 的 γ（步长）。
- ☐ 能算 GBDT 平方损失的 pseudo-residual、全局 γ、叶更新。
- ☐ 能默写 XGBoost 的 `w*=-G/(H+λ)` 和 Gain 公式，并代数字。

- ☐ 能解释 γ 和 λ 各自控制什么。
- ☐ 能背 LightGBM 的 leaf-wise、GOSS、EFB、categorical。
- ☐ 能背 XGBoost vs LightGBM 对比表。
- ☐ 能说出 20 条高频陷阱里的至少 15 条。
- ☐ 做一遍 §11 的 15 题，错题回到对应小节。

---

15. 一句话总结

> **Lecture 9**：单棵树用不纯度选分裂，但高 variance；**Bagging** 用 bootstrap + 平均降 variance；**Random Forest** 再加特征随机，进一步降低树间相关性。
>
> **Lecture 10**：**Boosting** 顺序纠错降 bias；**AdaBoost** 用样本权重 + 弱分类器权重实现；对噪声敏感。
>
> **Lecture 11**：**Gradient Boosting** 把 boosting 看成函数空间的梯度下降，拟合 pseudo-residual；**XGBoost** 加二阶 Taylor + 正则（γ、λ），用 `w*=-G/(H+λ)` 和 Gain 建树；**LightGBM** 用 leaf-wise + GOSS + EFB 加速。
>
> **Quiz 3 = 概念边界 + 6 个计算模板 + 20 条陷阱**；把 §10、§11、§12、§13 吃透，50 分钟足够。